NOME COMPLETO		
NOME COMPLETO		
LINK DO GITHUB		
LINK DO SITE		

Rubrica para avaliação de trabalhos			**Nota Maxima**	**Escala de Pontuação**					**Nota do componente**	**Nota do projeto** (Até 3 pontos)
Disciplinas	**Sprint**	**Descrição**		**Totalmente Satisfatório** (100%)	**Satisfatório** (75%)	**Parcialmente satisfatório** (50%)	**Pouco satisfatório** (25%)	**Insatisfatório** (Não cumpre o requisito)		
Arquitetura de Software	☐	Documentação Arquitetural: O que entregar: Diagramas iniciais C4 (Contexto e Containers). Registro de ADRs com decisões principais (ex.: monólito vs modulith). Documento simples com Quality Scenarios (ex.: desempenho, disponibilidade).	0,5	☑	☐	☐	☐	☐	0,5	**3,00**
	☐	Estilos Arquiteturais e DDD: O que entregar: Context Map com identificação de domínios e subdomínios. Definição de Bounded Contexts do projeto (ex.: agendamento, pagamentos). Lista de Entities, Value Objects e Aggregates para pelo menos 1 contexto.	1	☑	☐	☐	☐	☐	1	
	☐	Integração, APIs e Dados: O que entregar: Arquivo OpenAPI/Swagger com os principais endpoints do sistema. ADR justificando a escolha do banco de dados.	0,5	☑	☐	☐	☐	☐	0,5	
	☐	Atributos de Qualidade, Resiliência e Observabilidade: Documento com SLOs/SLIs (ex.: tempo de resposta, disponibilidade). Lista de estratégias de resiliência aplicadas (retry, timeout, circuit breaker). Plano simples de observabilidade (quais métricas e logs coletar).	0,5	☑	☐	☐	☐	☐	0,5	
	☐	Entrega Contínua: O que entregar: Arquivo pipeline CI/CD esqueleto (build, test, deploy). Estratégia de deploy definida (blue/green, rollback ou simples). Documento curto com runbook de incidentes (como agir se o sistema cair).	0,5	☑	☐	☐	☐	☐	0,5	
Back-end Avançado	☐	Cache Distribuído com Redis (Implementação do cache-aside para consultas frequentes; Uso de TTL para controlar expiração; Demonstração de invalidação de cache em updates)	1,00	☑	☐	☐	☐	☐	1	**3,00**
	☐	Mensageria e Assinatura de Eventos (Uso de Redis Pub/Sub para publicar e consumir eventos; Logs que comprovem a circulação de mensagens)	0,50	☑	☐	☐	☐	☐	0,5	
	☐	Uso adequado do Docker Compose para orquestração (Configuração correta de Nginx como proxy reverso)	1,00	☑	☐	☐	☐	☐	1	
	☐	Boas Práticas e Observabilidade (Organização do código (módulos, DTOs, services, controllers); Logs claros para identificar fluxos; Tratamento de erros)	0,50	☑	☐	☐	☐	☐	0,5	
Tech Forge	☐	**Gestão do Trabalho e Progresso via Jira**: Demonstração do uso eficaz do Jira para criar, organizar e gerenciar as tarefas do projeto, refletindo o fluxo de trabalho da equipe. Inclui a criação de issues (stories, tasks, bugs), atribuições e atualizações de status para garantir a transparência do progresso.	1	☑	☐	☐	☐	☐	1	**3,00**
	☐	**Análise do Contexto do Projeto com Cynefin e Escolha da Abordagem**: Aplicação do Framework Cynefin para classificar o projeto em um dos seus domínios (Simples, Complicado, Complexo, Caótico), justificando a escolha e, a partir dela, determinando a abordagem de gestão de projetos (preditiva, ágil ou híbrida) mais adequada para o seu desenvolvimento.	1	☑	☐	☐	☐	☐	1	
	☐	**Planejamento e Condução do Ciclo de Vida do Projeto:** - Para Projetos Preditivos: Demonstração da compreensão e aplicação das fases do ciclo de vida preditivo (Iniciação, Planejamento, Execução, Monitoramento e Controle, Encerramento) e da utilização eficaz do PM Canvas para estruturar e comunicar o plano do projeto. A clareza na definição e responsabilidades dos papéis de gestão de projetos (ex: Gerente de Projetos, Analistas) também é avaliada. - Para Projetos Ágeis/Scrum: Elaboração de um Product Backlog claro, priorizado e bem gerenciado, com itens descritos em formato de User Stories que contenham critérios de aceite bem definidos. Padronização e implementação de versões para lançamento de parciais do software. (Podendo se utilizar de Scrum ou Kanban)	1	☑	☐	☐	☐	☐	1	
Evolução do projeto	*NSA*									
	NSA									
	NSA									